Documento Tutorial Paso a Paso

Proceso de síntesis e instalación del sistema operativo creado con Yocto Project

BitBakers:
Alexandra Alfaro Elizondo
José Pablo Esquetini Fallas
Kendall Madrigal Campos
Taller de Sistemas Embebidos
Instituto Tecnológico de Costa Rica

I Semestre 2026

Resumen

Este documento describe paso a paso el proceso seguido para sintetizar e instalar una imagen personalizada de Linux utilizando Yocto Project, integrando una aplicación desarrollada en Rust con OpenCV. El objetivo es documentar de forma reproducible la preparación del entorno, la configuración de capas y recetas, la compilación de la imagen final y su ejecución en una máquina virtual mediante VirtualBox.

Índice

1. Introducción **3**
- 1.1. Objetivo general . . . 3
- 1.2. Objetivos específicos . . . 3

2. Descripción general de la solución **3**
- 2.1. Aplicación desarrollada . . . 3
- 2.2. Arquitectura general . . . 4

3. Requisitos previos **5**
- 3.1. Software requerido . . . 5
- 3.2. Versiones utilizadas . . . 6

4. Preparación del entorno de trabajo **6**
- 4.1. Instalación de dependencias del host . . . 6
- 4.2. Obtención del código fuente . . . 6
- 4.3. Inicialización del entorno Yocto . . . 7

5. Configuración del proyecto Yocto **7**
- 5.1. Configuración de `local.conf` . . . 7
- 5.2. Configuración de `bblayers.conf` . . . 8

6. Proceso de síntesis de la imagen **9**
- 6.1. Comando de compilación . . . 9

7. Archivos generados **9**

8. Instalación y prueba en VirtualBox **9**
- 8.1. Creación de la máquina virtual . . . 9
- 8.2. Carga de la imagen . . . 11

9. Páginas consultadas **13**

1. Introducción

Este tutorial presenta el procedimiento utilizado para construir una imagen personalizada de Linux orientada a la ejecución de una aplicación multimedia desarrollada en Rust con soporte de OpenCV. La solución fue generada utilizando Yocto Project para arquitectura x86 y validada en una máquina virtual con VirtualBox.

1.1. Objetivo general

Documentar el proceso de síntesis e instalación de una imagen Linux personalizada, reproducible y funcional, que incluya la aplicación desarrollada en Rust y sus dependencias.

1.2. Objetivos específicos

- Preparar el entorno de desarrollo requerido para Yocto Project.
- Configurar las capas necesarias para compilar una imagen mínima personalizada.
- Integrar la aplicación en Rust con sus dependencias.
- Generar la imagen del sistema operativo para arquitectura x86.
- Instalar y ejecutar la imagen en VirtualBox.

2. Descripción general de la solución

En esta sección se resume qué hace la aplicación y cómo se integra dentro de la imagen Linux.

2.1. Aplicación desarrollada

La aplicación desarrollada, titulada **Detector de Estado Ocular en Video con Rust, OpenCV y Yocto Project**, consiste en un sistema de procesamiento de video orientado a detectar el estado de apertura o cierre de los ojos de una persona a lo largo de una secuencia de video. Su propósito principal es analizar cuadros de video de manera automática para identificar el rostro, localizar puntos de referencia faciales y estimar el estado ocular de cada ojo, generando como salida un video procesado con información visual superpuesta.

- **Nombre del proyecto:** Detector de Estado Ocular en Video con Rust, OpenCV y Yocto Project.

- **Funcionalidad principal:** La aplicación procesa un video de entrada, detecta el rostro principal en cada cuadro y localiza puntos característicos de la cara mediante técnicas de visión por computadora. A partir de estos puntos, calcula métricas geométricas y visuales para determinar si el ojo izquierdo y el ojo derecho se encuentran abiertos o cerrados. Además, dibuja sobre el video las regiones de interés, etiquetas de estado y una gráfica del comportamiento temporal del indicador ocular, y finalmente genera un video de salida con el resultado del análisis.

- **Casos de uso:**

  - Detección de parpadeo mientras la persona sonríe, para evaluar la robustez del sistema ante cambios de expresión facial.
  - Prueba de código morse con parpadeos, utilizando la secuencia de apertura y cierre de ojos como patrón observable en video.
  - Prueba con lentes, para analizar el desempeño del sistema cuando existen elementos adicionales sobre el rostro que pueden afectar la detección visual.

- **Flujo general de procesamiento de imágenes o video:** El sistema inicia con la lectura de un video de entrada cuadro por cuadro. Cada cuadro es convertido a escala de grises y ecualizado para mejorar la detección facial. Posteriormente, se localiza el rostro principal mediante un clasificador Haar Cascade y se estiman landmarks faciales usando un modelo Facemark LBF. Con base en los puntos correspondientes a ambos ojos, se construyen regiones de análisis y se calcula el indicador *Eye Aspect Ratio* (EAR), complementado con métricas visuales de la zona del párpado. Finalmente, se determina el estado de cada ojo, se añaden elementos gráficos al cuadro procesado y se almacena el resultado en un video de salida.

2.2. Arquitectura general

La arquitectura general de la solución se compone de un entorno de desarrollo en el computador host, un sistema de construcción basado en Yocto Project, una aplicación multimedia implementada en Rust con soporte de OpenCV y una máquina virtual de destino donde se valida el funcionamiento de la imagen generada. Cada uno de estos componentes cumple un papel específico dentro del flujo de construcción, integración y prueba del sistema.

- **Host de desarrollo:** El host de desarrollo corresponde al computador principal utilizado para preparar el entorno de trabajo, editar el código fuente, configurar Yocto Project, compilar la imagen del sistema operativo y realizar pruebas preliminares de la aplicación. En este entorno también se gestionan las dependencias necesarias, los repositorios del proyecto y los archivos de configuración del proceso de construcción.

- **Entorno Yocto:** El entorno Yocto constituye la base del proceso de síntesis del sistema operativo. A través de sus herramientas de construcción se define una imagen Linux personalizada para arquitectura x86, incorporando únicamente los componentes requeridos por la aplicación. Este entorno permite automatizar la compilación, integración de paquetes, resolución de dependencias y generación reproducible de la imagen final.

- **Capas utilizadas:** La solución hace uso de un conjunto de capas de Yocto que organizan la configuración y las recetas del sistema. Entre ellas se incluyen el "poky", para compilar usando Yocto Project, "meta-rust-bin"para agregar la compatibilad del lenguaje Rust, "meta-openembedded"para agregar bibliotecas básicas como python y manejo de gráficos así como OpenCV, "meta-clang"para compilar la aplicación de Rust y finalmente, el propio meta de la aplicación realizada.

- **Aplicación Rust:** La aplicación principal fue desarrollada en el lenguaje Rust y se encarga del procesamiento de video para la detección del estado ocular. Su implementación contempla la lectura de un video de entrada, el análisis cuadro por cuadro, la detección facial, la estimación de landmarks y la clasificación del estado de los ojos. Posteriormente, genera un video de salida con anotaciones visuales que permiten observar el resultado del procesamiento.

- **Bibliotecas OpenCV:** OpenCV se utiliza como biblioteca base para las funciones de visión por computadora requeridas por la aplicación. Entre sus aportes se encuentran la captura y escritura de video, la conversión y mejora de imágenes, la detección de rostros y el uso de modelos faciales para la localización de puntos característicos. La integración entre Rust y OpenCV se realiza mediante bindings que permiten acceder desde Rust a las funcionalidades implementadas originalmente en C++.

- **Máquina virtual destino:** La imagen generada con Yocto Project se ejecuta en una máquina virtual configurada en VirtualBox con arquitectura x86. Este entorno de destino permite validar el arranque del sistema, comprobar la instalación correcta de las dependencias y demostrar la ejecución funcional de la aplicación dentro de un sistema operativo personalizado.

3. Requisitos previos

3.1. Software requerido

- Sistema operativo host.

- Git.

- Dependencias de compilación de Yocto.

- VirtualBox.
- Rust/Cargo.
- OpenCV.

3.2. Versiones utilizadas

Herramienta / componente	**Versión**
Yocto Project / Poky	Kirkstone
Rust	1.94.0 (entorno de compilación), con versión mínima requerida 1.75
Cargo	1.94.0
OpenCV	4.6.0
VirtualBox	7.2.6r172322
Sistema operativo host	Ubuntu 24.04.4 LTS

4. Preparación del entorno de trabajo

4.1. Instalación de dependencias del host

En primer lugar, se debe establecer el ambiente, para esto corra los siguientes comandos, con lo cual se instalará lo necesario, así como hacer la carpeta donde se trabajará.

```
sudo apt update && sudo apt upgrade -y

sudo apt install -y \
  gawk wget git diffstat unzip texinfo gcc build-essential \
  chrpath socat cpio python3 python3-pip python3-pexpect \
  xz-utils debianutils iputils-ping python3-git python3-jinja2 \
  libegl1-mesa libsdl1.2-dev pylint xterm python3-subunit \
  mesa-common-dev zstd liblz4-tool file locales libacl1 \
  clang llvm

sudo locale-gen en_US.UTF-8

mkdir -p ~/yocto
cd ~/yocto
```

Listing 1: Instalación de dependencias en el host

4.2. Obtención del código fuente

Ahora en la carpeta creada, se instalarán todos los meta-layers necesarios.

```
# Poky (Kirkstone)
git clone -b kirkstone git://git.yoctoproject.org/poky.git

# meta-openembedded (Kirkstone)
git clone -b kirkstone https://github.com/openembedded/meta-
   openembedded.git

# meta-rust-bin
git clone -b master https://github.com/rust-embedded/meta-rust-
   bin.git

# meta-clang (Kirkstone)
git clone -b kirkstone https://github.com/kraj/meta-clang.git

# meta-myapp (detector_ojos)
git clone -b meta-myapp https://github.com/LexieVaneska/
   Proyecto_1_Embebidos.git meta-myapp
```

Listing 2: Clonado de repositorios

4.3. Inicialización del entorno Yocto

Ahora se inicializará el ambiente de Yocto.

```
cd ~/yocto
source poky/oe-init-build-env build
```

Listing 3: Inicialización del entorno Yocto

5. Configuración del proyecto Yocto

Ahora en la carpeta de build se deben configurar el archivo **local.conf** y **bblayers.conf**

```
cd yocto/build
```

Listing 4: Ejemplo de parámetros relevantes en local.conf

5.1. Configuración de `local.conf`

Cambie el contenido del archivo preterminado por el siguiente.

```
MACHINE = "genericx86-64"

IMAGE_FSTYPES += "wic.vmdk"
```

```
DISTRO_FEATURES:append = "␣x11␣opengl"

RUST_VERSION = "1.78.0"

PACKAGECONFIG:append:pn-opencv = "␣gtk"

PACKAGECONFIG:append:pn-opencv = "␣dnn"
EXTRA_OECMAKE:append:pn-opencv = "␣-DBUILD_opencv_objdetect=ON␣-
   DBUILD_opencv_face=ON"

LICENSE_FLAGS_ACCEPTED += "commercial"

BB_NO_NETWORK = "0"
SRCREV_pn-myapp = "${AUTOREV}"

EXTRA_IMAGE_FEATURES += "debug-tweaks"

BB_NUMBER_THREADS = "8"
PARALLEL_MAKE = "-j␣8"

EXTRA_IMAGE_FEATURES += "ssh-server-openssh"
```

Listing 5: Ejemplo de parámetros relevantes en local.conf

5.2. Configuración de `bblayers.conf`

Ahora haga lo mismo con el contenido del bblayers.conf, copiando el siguiente contenido.

```
MPOKY_BBLAYERS_CONF_VERSION = "2"

BBPATH = "${TOPDIR}"
BBFILES ?= ""

BBLAYERS ?= "␣\
␣␣/home/jpesquetinif/yocto/poky/meta␣\
␣␣/home/jpesquetinif/yocto/poky/meta-poky␣\
␣␣/home/jpesquetinif/yocto/poky/meta-yocto-bsp␣\
␣␣/home/jpesquetinif/yocto/meta-openembedded/meta-oe␣\
␣␣/home/jpesquetinif/yocto/meta-openembedded/meta-python␣\
␣␣/home/jpesquetinif/yocto/meta-openembedded/meta-multimedia␣\
␣␣/home/jpesquetinif/yocto/meta-rust-bin␣\
␣␣/home/jpesquetinif/yocto/meta-myapp␣\
```

```
␣␣/home/jpesquetinif/yocto/meta-clang␣\
␣␣"
```

Listing 6: Ejemplo de parámetros relevantes en bblayers.conf

6. Proceso de síntesis de la imagen

6.1. Comando de compilación

Una vez configurados todos los archivos se procede a realizar la creación de la imagen.

```
bitbake myapp-image
```

Listing 7: Compilación de la imagen

7. Archivos generados

Una vez que se complete la compilación, se puede encontrar la imagen generada con los siguientes comandos.

```
ls ~/yocto/build/tmp-glibc/deploy/images/genericx86-64/*.wic.vmdk
```

Listing 8: Ruta típica de artefactos generados

Se puede ver que la imagen se encuentra dentro de la cerpeta build ->tmp-glibc ->deploy ->images ->genericx86.

Aquí se encuentran 2 tipos de imagen, la que se va a utilizar el la imagen simple, ignorar la .rootfs.

8. Instalación y prueba en VirtualBox

8.1. Creación de la máquina virtual

En Virtual Box cree una máquina con las siguientes especificaciones.

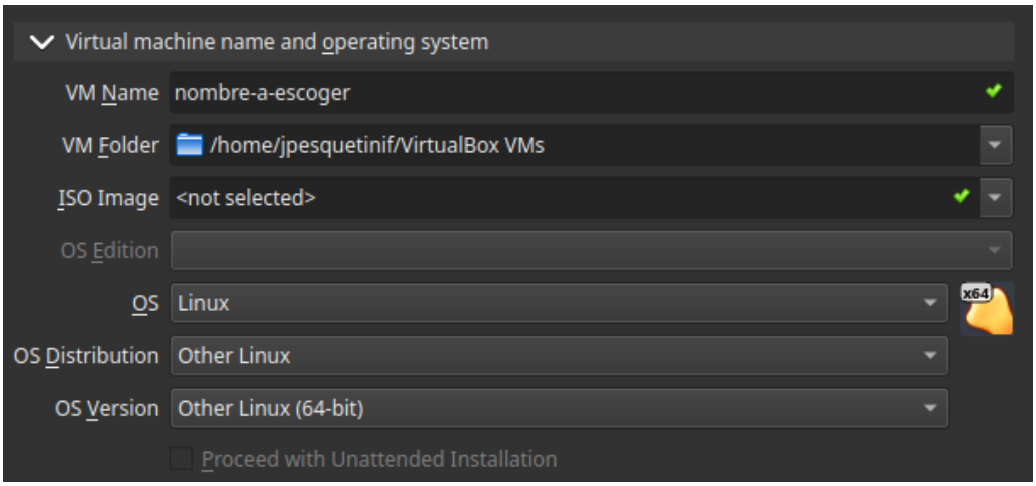

Figura 1: Configuración básica de la VM

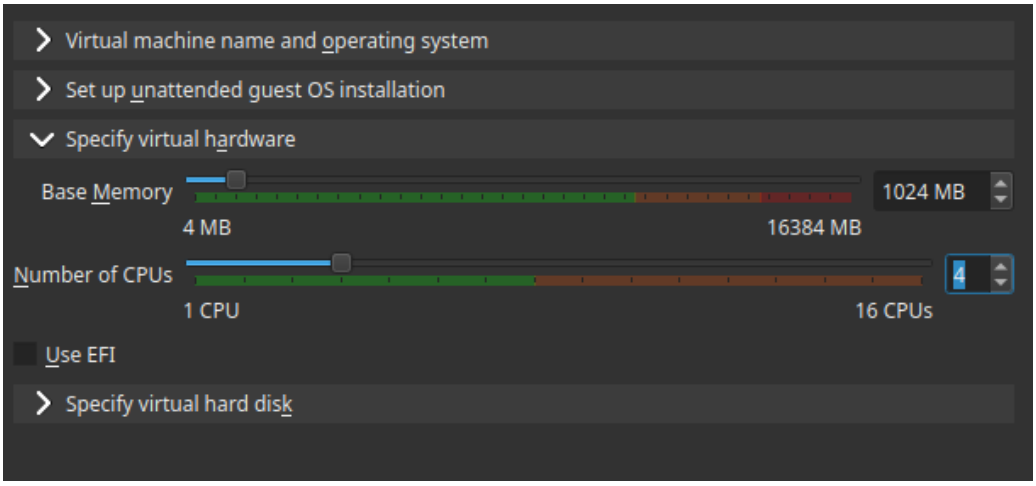

Figura 2: Configuraciones del sistema

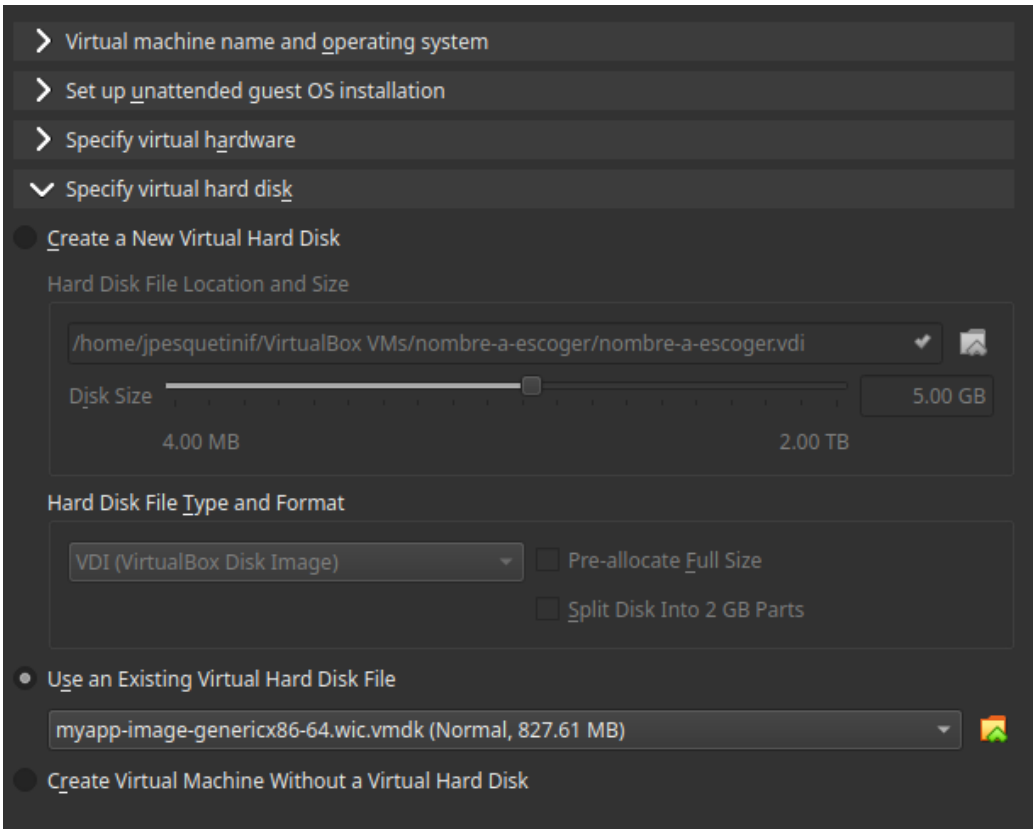

Figura 3: Configuraciones del

8.2. Carga de la imagen

Ahora se debe cargar la imagen a la VM.

Para esto debe irse a los ajustes, en Almacenamiento, subir el archivo encontrado previamente a la parte de controller:SATA, y finalmente en sistema activar la opción de UEFI. Como se muestra a continuación.

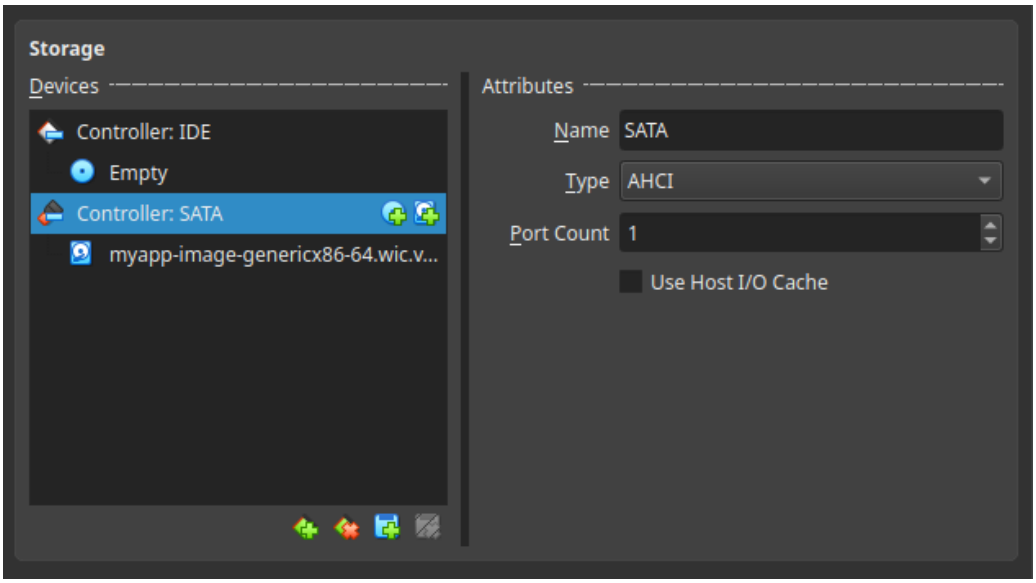

Figura 4: Configuraciones del almacenamiento

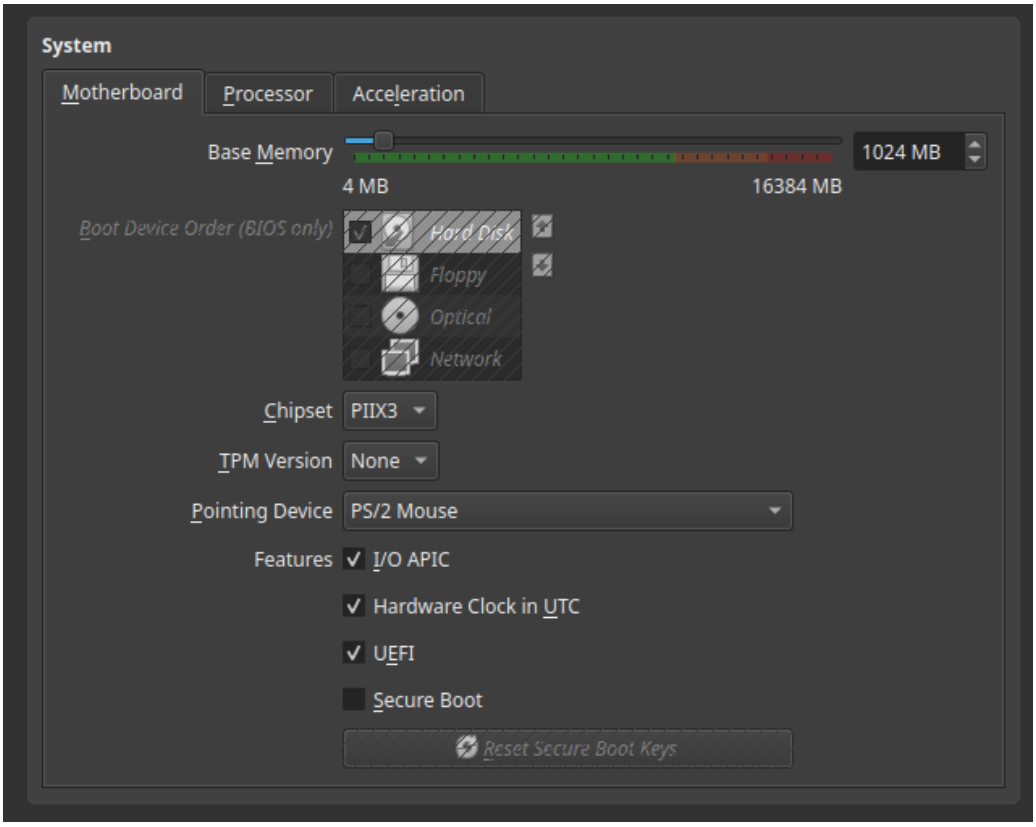

Figura 5: Configuraciones del sistema

Con esto solo falta correr el programa. Para esto debe encender la máquina y usar los siguientes comandos.

```
root
```

```
# No se ocupa contrasena

export DISPLAY=:0
xinit &

cd /usr/share/myapp
detector_ojos
```

Listing 9: Ruta típica de artefactos generados

9. Páginas consultadas

Durante el desarrollo del proyecto se consultaron distintas fuentes de documentación técnica para comprender el uso de Rust, OpenCV y la integración entre ambas tecnologías. A continuación se listan las principales páginas utilizadas como referencia:

- Rust Programming Language. https://www.rust-lang.org/
- Documentación oficial de Cargo. https://doc.rust-lang.org/cargo/
- Crate `opencv` en Rust. https://crates.io/crates/opencv
- Documentación del crate `opencv`. https://docs.rs/opencv/
- OpenCV oficial. https://opencv.org/
- Documentación oficial de OpenCV. https://docs.opencv.org/
- MediaPipe Solutions Guide. https://ai.google.dev/edge/mediapipe/solutions/guide?hl=es-419